

Machine Learning

Unit-III (A): Support Vector Machine (SVM)

Dr. Satveer Singh
Assistant Professor,
JK Lakshmipat University, Jaipur.

Support Vector Machine (SVM) Algorithm

- SVM is a supervised machine learning algorithm used for classification and regression tasks.
- It tries to find the best boundary known as hyperplane that separates different classes in the data.
- It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.
- The main goal of SVM is to maximize the margin between the two classes.
- The larger the margin the better the model performs on new and unseen data.

Key Concepts of Support Vector Machine

1. Hyperplane

- A **decision boundary** separating different classes in feature space and is represented by the equation $wx + b = 0$ in linear classification.

2. Support Vectors

- The **closest data points to the hyperplane**, crucial for determining the hyperplane and margin in SVM.

3. Margin

- The **distance between the hyperplane and the support vectors**.
- SVM aims to maximize this margin for better classification performance.

Key Concepts of Support Vector Machine

4. Kernel

- A function that maps data to a higher-dimensional space enabling SVM to handle non-linearly separable data.

5. Hard Margin

- A maximum-margin hyperplane that perfectly separates the data without misclassifications.

6. Soft Margin

- Allows some misclassifications by introducing slack variables, balancing margin maximization and misclassification penalties when data is not perfectly separable.

Key Concepts of Support Vector Machine

7. C

- A regularization term balancing margin maximization and misclassification penalties.
- A higher C value forces stricter penalty for misclassifications.

8. Hinge Loss

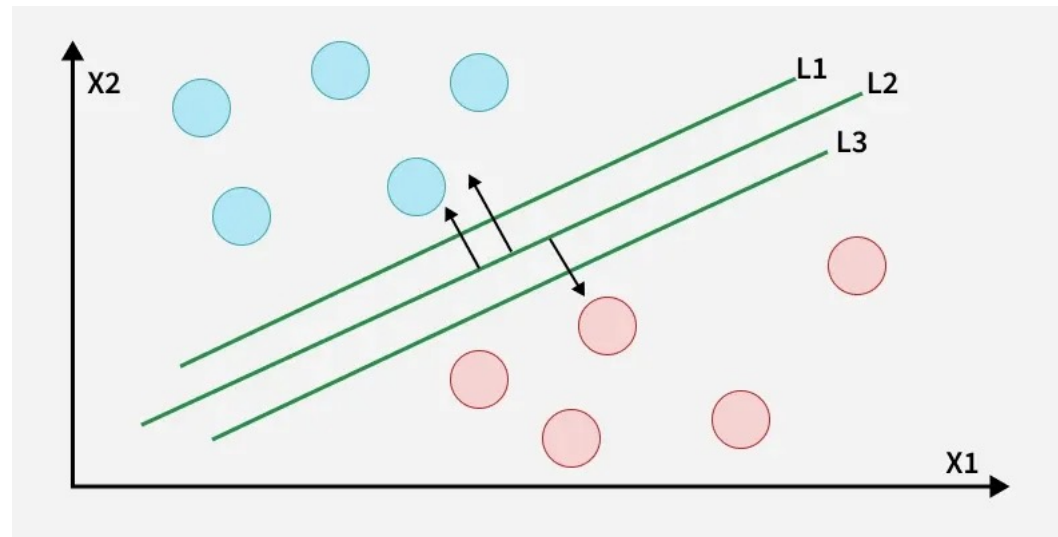
- A loss function penalizing misclassified points or margin violations and is combined with regularization in SVM.

9. Dual Problem

- Involves solving for Lagrange multipliers associated with support vectors, facilitating the kernel trick and efficient computation.

How does Support Vector Machine Algorithm Work?

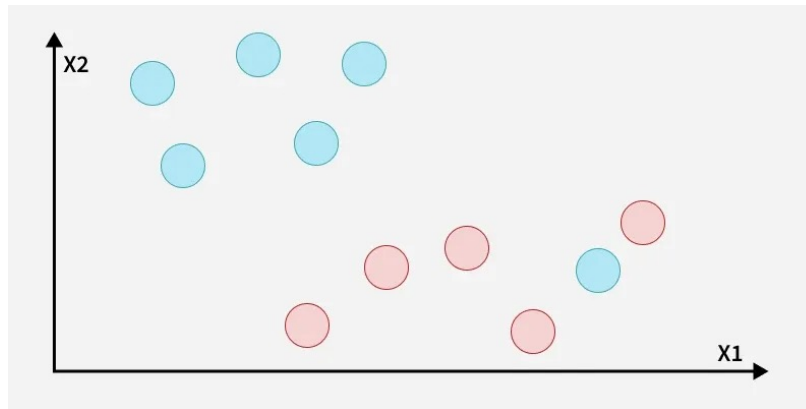
- The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by **maximizing the margin** between them.
- This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side.



Multiple hyperplanes separate the data from two classes

How does Support Vector Machine Algorithm Work?

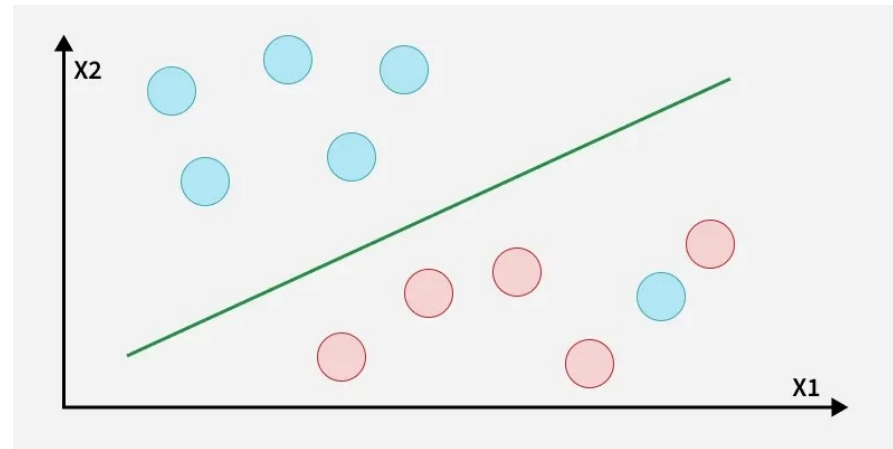
- The **best hyperplane also known as the "hard margin"** is the one that maximizes the distance between the hyperplane and the nearest data points from both classes.
- This ensures a clear separation between the classes.
- So from the last slide figure, we choose L2 as hard margin.
- Let's consider a scenario like shown below:



Selecting hyperplane for data with outlier

How does SVM classify the data?

- Here, we have one blue ball in the boundary of the red ball.
- The blue ball in the boundary of red ones is an outlier of blue balls.
- The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin.
- SVM is robust to outliers.



Hyperplane which is the most optimized one

How does SVM classify the data?

- A soft margin allows for some misclassifications or violations of the margin to improve generalization.
- The SVM optimizes the following equation to balance margin maximization and penalty minimization:

$$\text{Objective Function} = \left(\frac{1}{\text{margin}} \right) + \lambda \sum \text{penalty}$$

Here,

$$\text{Minimize} = \underbrace{\frac{1}{\text{margin}}}_{\text{Maximize margin}} + \underbrace{\lambda \sum \text{penalty}}_{\text{reduce classification errors}}$$

How does SVM classify the data?

- λ is a **regularization parameter** that controls the **trade-off** between:
 - Margin size
 - Classification errors
- **Case 1: Large λ**
 - Misclassification penalty becomes important.
 - Model tries to **reduce errors** even if margin becomes smaller.
- **Case 2: Small λ**
 - Margin maximization becomes more important.
 - Model may **allow some errors** to keep a large margin.
- The **penalty used for violations is often hinge loss** which has the following behaviour:
 - If a data point is **correctly classified and within the margin** there is no penalty (loss = 0).
 - If a point is incorrectly classified or violates the margin the **hinge loss increases proportionally to the distance of the violation**.

What if data is not linearly separable?

- When data is not linearly separable, i.e., it can't be divided by a straight line, SVM uses a technique called kernels to map the data into a higher-dimensional space where it becomes separable.
- This transformation helps SVM find a decision boundary even for non-linear data.

Kernels

- A kernel is a function that maps data points into a higher-dimensional space without explicitly computing the coordinates in that space.
- This allows SVM to work efficiently with non-linear data by implicitly performing the mapping.
- For example consider data points that are not linearly separable.
- By applying a kernel function SVM transforms the data points into a higher-dimensional space where they become linearly separable.

Kernels in SVM

Some of the ideas behind kernels in SVM are:

1. **Similarity Measurement:** Kernels calculate how closely two data points are related in feature space.
2. **Dimensional Expansion:** They map data into higher dimensions implicitly without expensive computation.
3. **Flexible Boundaries:** They enable the model to draw curved or irregular separation lines.
4. **Efficient Computation:** They avoid manually engineering additional feature columns.
5. **Inner Product Trick:** Kernels compute complex feature interactions using simple mathematical shortcuts.

Using Kernel Functions

Some of the reasons to use kernel functions are:

1. **Non-Linear Patterns:** Many real-world problems cannot be separated with straight lines.
2. **Smooth Separation:** Kernels allow boundaries that curve around data clusters.
3. **Reduced Feature Work:** They eliminate manual creation of polynomial or cross terms.
4. **Improved Accuracy:** Non-linear transformations capture deeper relationships.
5. **General Adaptability:** Kernels make SVM work well across diverse data types.

Mathematical Formula

- Mathematically, SVM Kernel Function can be represented by:

$$K(x, x') = \phi(x)^T \phi(x')$$

- Means,

Kernel value = dot product of transformed feature vectors

- Where,
 - x, x' : Two feature vectors
 - $\phi(\cdot)$: Mapping function to higher-dimensional space
 - $\phi(x)^T \phi(x')$: Dot product measuring similarity after mapping
 - $K(x, x')$: Kernel similarity score
- Instead of computing $\phi(x)$ explicitly which can be expensive. We compute the similarity directly using K , which is faster. This is called the kernel trick.
- Let's understand using an example:

Step 1: Take Two Data Points

- Suppose we have two feature vectors:

$$x = (1, 2)$$

$$x' = (3, 4)$$

- These are **2-dimensional vectors**.

Step 2: Define a Mapping Function

- Assume the mapping function transforms the data into 3-dimensional space:
- $\phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$

Step 3: Compute the Mapping Explicitly

- For $x = (1, 2)$

$$\phi(x) = (1^2, 2^2, \sqrt{2} \ 1 * 2)$$

$$\phi(x) = (1, 4, 2\sqrt{2})$$

- For $x' = (3, 4)$

$$\phi(x') = (3^2, 4^2, \sqrt{2} \ 3 * 4)$$

$$\phi(x') = (9, 16, 12\sqrt{2})$$

- Now both vectors are 3-dimensional.

Step 4: Compute Dot Product in Higher Dimension

$$\phi(x)^T \phi(x') = (1)(9) + (4)(16) + (2\sqrt{2})(12\sqrt{2})$$

- Calculate each term:

$$1 \times 9 = 9$$

$$4 \times 16 = 64$$

$$2\sqrt{2} \times 12\sqrt{2} = 24 \times 2 = 48$$

- Now add them:

$$9 + 64 + 48 = 121$$

- So,

$$\phi(x)^T \phi(x') = 121$$

Step 5: Use the Kernel Function Instead

- Instead of computing the mapping, we use the Polynomial Kernel:

$$K(x, x') = (x \cdot x')^2$$

- First compute the dot product in original space:

$$x \cdot x' = (1)(3) + (2)(4) = 3 + 8 = 11$$

- Now apply the kernel:

$$K(x, x') = 11^2$$

$$K(x, x') = 121$$

Step 6: Compare Results

Method	Result
High-dimensional dot product	121
Kernel function	121

Linear Kernel

- Measures similarity using a **simple dot product**; best for linearly separable data with fast computation.
- **Formula:**

$$K(x, x') = x^\top x'$$

- Where,
 - x, x' : Feature vectors
 - $x^\top x'$: Similarity between the two points

Characteristics

- **Dot Product Similarity:** It evaluates similarity using a straightforward dot product between feature vectors.
- **Straight Boundaries:** It works best when the data is linearly separable.
- **Fast Training:** It trains faster compared to more complex kernels.
- **Sparse Features:** It is commonly used for text or document classification tasks.
- **Wide Margins:** It produces boundaries that remain clean and interpretable.

Polynomial Kernel

- Captures more complex relationships by raising feature interactions to a power, allowing curved decision boundaries.

- **Formula:**

$$K(x, x') = (x^\top x' + r)^d$$

- Where,
 - $x^\top x'$: Dot product similarity
 - r : Bias term
 - d : Degree of polynomial
- The bias term r is a constant added to the dot product in the polynomial kernel to make the model more flexible and allow additional feature interactions.

Characteristics

- **Feature Interactions:** It represents combinations of original features up to a selected power.
- **Degree Control:** Higher degrees create more twists and turns in the decision boundary.
- **Smooth Curvature:** It handles patterns that bend gently across feature space.
- **Overfitting Risk:** Improper degree settings may cause the model to memorize noise.
- **Adjustable Shape:** Coefficients allow fine-grained boundary customization.

Radial Basis Function (RBF) or Gaussian Kernel

- Maps data into infinite dimensions and works well when class boundaries are highly non-linear and irregular.

- **Formula:**

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

- This formula calculates the similarity between two data points based on their distance.
- Where,
 - *exp*: Exponential curve function which converts the distance value into a similarity score between 0 and 1.
 - γ : Controls how far influence of a point spreads
 - Small $\gamma \rightarrow$ wide smooth curves
 - Large $\gamma \rightarrow$ very tight boundaries around points
 - $\|x - x'\|^2$: Squared Euclidean distance between two points

Characteristics

- **Distance-Based Similarity:** Closer points are treated as more alike, forming tight clusters.
- **Flexible Curves:** It adapts to boundaries that wrap and spiral around complex shapes.
- **Gamma Influence:** Higher gamma values make the decision boundary more sensitive.
- **Default Choice:** It performs well when data complexity is unknown.
- **Smooth Output:** It constructs continuous and visually appealing separation surfaces.

Step 1: Take Two Data Points

- Let

$$x = (1, 2)$$

$$x' = (3, 4)$$

- Assume

$$\gamma = 0.5$$

Step 2: Compute Squared Distance

$$\|x - x'\|^2 = (1 - 3)^2 + (2 - 4)^2$$

$$\|x - x'\|^2 = (-2)^2 + (-2)^2 = 4 + 4$$

$$\|x - x'\|^2 = 8$$

Step 3: Multiply by Gamma

$$-\gamma \|x - x'\|^2 = -(0.5)(8) = -4$$

Step 4: Apply Exponential Function

$$K(x, x') = e^{-4}$$

$$e^{-4} \approx 0.0183$$

Step 5: Final Kernel Similarity

$$K(x, x') \approx 0.018$$

Interpretation

- The RBF kernel value ranges between 0 and 1.

Distance	Kernel Value	Meaning
very small	close to 1	points are very similar
large	close to 0	points are very different

- Since 0.018 is close to 0, the two points are not very similar.

Comparison Table

Kernel	Boundary Shape	Speed	Best For
Linear	Straight line	Fast	Linearly separable data
Polynomial	Curved	Medium	Feature interactions
RBF	Very flexible	Slower	Highly non-linear data

Choosing Right Kernel

Some of the scenarios for choosing right kernel are:

1. **Linear Data:** Use the linear kernel when boundaries remain straight and wide.
2. **Curved Trends:** Select the polynomial kernel to capture mild curvature.
3. **Complex Shapes:** Prefer the RBF kernel for spiralling or circular behaviour.

Applications

- **Image Recognition:** Kernels detect textures, edges and fine visual variations, making them useful for classifying objects and visual patterns.
- **Text Classification:** Linear kernels handle sparse language features efficiently, helping categorize documents, emails, and sentiment-based text.
- **Medical Diagnosis:** RBF kernels identify biological irregularities that may indicate disease, supporting early detection and classification.
- **Traffic Anomaly Detection:** Kernels highlight suspicious network behaviour, helping cybersecurity systems identify unauthorized access patterns.
- **Financial Fraud Analysis:** Kernel boundaries expose unusual payment clusters and transaction behaviours, assisting fraud detection systems.

Advantages

- **Non-Linear Handling:** They allow the model to learn from complex data patterns that cannot be separated by straight lines, enabling more accurate classification boundaries.
- **Feature Flexibility:** They reduce the need for manually crafted feature transformations, since kernels implicitly create richer dimensional representations.
- **Noise Resistance:** They perform well in moderately noisy environments, helping the model remain stable even when data contains irregularities.
- **Smooth Boundaries:** They generate visually consistent separation surfaces that curve naturally around data clusters and reduce jagged decision lines.
- **Domain Versatility:** They work across text, images, signals and medical data, making them suitable for a wide range of machine learning applications.

Mathematical Computation of SVM

- Consider a binary classification problem with two classes, labelled as +1 and -1.
- We have a training dataset consisting of input feature vectors X and their corresponding class labels Y .
- The equation for the linear hyperplane can be written as:

$$w^T x + b = 0 \rightarrow \text{line or plane or hyperplane}$$

Where:

- w is the normal vector to the hyperplane (the direction perpendicular to it) and it controls the direction of the boundary.
- b is the offset or bias term representing the distance of the hyperplane from the origin along the normal vector w .

Distance from a Data Point to the Hyperplane

- The distance between a data point x_i and the decision boundary can be calculated as:

$$d_i = \frac{w^T x_i + b}{||w||}$$

- It gives the signed perpendicular distance between a data point and the decision boundary.
- Here,
 - $w^T x_i + b$ measures how far the point is from the boundary along the direction of w .
 - But this value alone is not the true geometric distance, because it depends on the magnitude of w .
 - $||w|| = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$, it represents the length of the weight vector or Euclidean norm of the weight vector.
- Dividing by $||w||$ normalizes the distance, giving the true perpendicular distance.

Linear SVM Classifier

- Distance from a Data Point to the Hyperplane:

$$\hat{y} = \begin{cases} 1 & : w^T x + b \geq 0 \\ -1 & : w^T x + b < 0 \end{cases}$$

- Where $\hat{y}$ is the predicted label of a data point.

Example

Suppose

$$w = [3, 4], b = -10$$

Point:

$$x_i = [2, 1]$$

Step 1: Compute $w^T x_i + b$

$$(3 \times 2 + 4 \times 1) - 10 = 6 + 4 - 10 = 0$$

Step 2: Compute $\|w\|$

$$\|w\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = 5$$

Step 3: Distance

$$d_i = \frac{0}{5} = 0$$

- So the **point lies exactly on the decision boundary**.

Margin

- Decision Boundary (Hyperplane):

$$w^T x + b = 0$$

- Positive margin:

$$w^T x + b = 1$$

- Negative margin:

$$w^T x + b = -1$$

- Distance Between Parallel Hyperplanes if:

$$w^T x + b = c_1 \text{ and } w^T x + b = c_2$$

$$D_i = \frac{|c_1 - c_2|}{\|w\|}$$

Margin

- Here,

$$\begin{aligned}c_1 &= 1 \\c_2 &= -1\end{aligned}$$

- So distance becomes

$$D_i = \frac{|1 - (-1)|}{\|w\|} = \frac{2}{\|w\|}$$

- Margin between two classes will be:

$$\text{Margin} = \frac{2}{\|w\|}$$

- So, to maximize the margin, we need to **minimize** the $\|w\|$.

Optimization Problem for Hard Margin Linear SVM

- For a linearly separable dataset the goal is to find the hyperplane that maximizes the margin between the two classes while ensuring that all data points are correctly classified.
- This leads to the following optimization problem:

$$\underset{w,b}{\text{minimize}} \frac{1}{2} ||w||^2$$

- Subject to the constraint:

$$y_i(w^T x_i + b) \geq 1 \text{ for } i = 1, 2, 3, \dots, m$$

- Where,
 - y_i is the class label (+1 or -1) for each training instance.
 - x_i is the feature vector for the i^{th} training instance.
 - m is the total number of training instances.
 - $||w||$ is the Euclidean norm (length) of the weight vector.

Optimization Problem for Hard Margin Linear SVM

- The condition $y_i(w^T x_i + b) \geq 1$ ensures that each data point is correctly classified and lies outside the margin.
- Why use $\frac{1}{2} \|w\|^2$ instead of $\|w\|$?
 1. Mathematical convenience
 2. Always positive

Optimization Problem for Soft Margin Linear SVM

- In the presence of outliers or non-separable data the SVM allows some misclassification by introducing slack variables ζ_i .
- The optimization problem is modified as:

$$\underset{w,b}{\text{minimize}} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \zeta_i$$

- Subject to the constraints:

$$y_i(w^T x_i + b) \geq 1 - \zeta_i \text{ and } \zeta_i \geq 0 \text{ for } i = 1, 2, \dots, m$$

- Where:
 - C is a regularization parameter that controls the trade-off between margin maximization and penalty for misclassifications.
 - ζ_i are slack variables that represent the degree of violation of the margin by each data point.

Meaning of $C \sum_{i=1}^m \zeta_i$

- This is the penalty for margin violations.
 - If many points violate the margin $\rightarrow$ penalty increases
 - If violations are small $\rightarrow$ penalty is small
- C controls the trade-off between margin size and classification errors.

Value of C	Behaviour
Large C	Penalizes errors strongly $\rightarrow$ fewer misclassifications but smaller margin
Small C	Allows more misclassification $\rightarrow$ larger margin

- So C balances: Margin maximization vs classification accuracy

Meaning of $C \sum_{i=1}^m \zeta_i$

$$y_i(w^T x_i + b) \geq 1 - \zeta_i$$

- This is a relaxed version of the hard-margin constraint.
- Interpretation of Slack Variable ζ_i :

Value of ζ_i	Meaning
$\zeta_i = 0$	Correctly classified and outside margin
$0 < \zeta_i < 1$	Correctly classified but inside margin
$\zeta_i > 1$	Misclassified point

Dual Optimization Problem of SVM

- In SVM, the **original (primal) optimization problem** tries to find the best hyperplane:
 - maximize the **margin**
 - while correctly classifying training points.
- The primal objective is:

$$\underset{w,b}{\text{minimize}} \frac{1}{2} \| w \|^2$$

- Subject to

$$y_i(w^T x_i + b) \geq 1$$

- However, solving optimization problems with constraints directly is difficult.
- So, we convert it into an unconstrained optimization problem using **Lagrange multipliers**.

Idea of Lagrange Multipliers

- For every constraint, we introduce a new variable called a **Lagrange multiplier**.
- For the i^{th} constraint we introduce:

$$\alpha_i$$

- Where,

$$\alpha_i \geq 0$$

- Each α_i represents how strongly the constraint affects the solution.
 - If a constraint is **important (tight)** $\rightarrow \alpha_i > 0$
 - If a constraint is **not important** $\rightarrow \alpha_i = 0$
- Later these important points become **support vectors**.

Converting the Constraint into Standard Form

- Optimization methods usually express constraints in the form:

$$g(x) \geq 0$$

- So we rearrange:

$$y_i(w^T x_i + b) \geq 1$$

- Subtract 1:

$$y_i(w^T x_i + b) - 1 \geq 0$$

- Now it is in the required form.

Using Constraint in the Lagrangian

- The Lagrangian for inequality constraints is written as:

$$L = \frac{1}{2} \| w \|^2 - \sum \alpha_i g_i(x)$$

- Where,

$$g_i(x) = y_i(w^T x_i + b) - 1$$

- Thus, the SVM Lagrangian becomes:

$$L(w, b, \alpha) = \frac{1}{2} \| w \|^2 - \sum_{i=1}^m \alpha_i [y_i(w^T x_i + b) - 1]$$

Apply Karush–Kuhn–Tucker (KKT) Conditions

- To find the optimal solution, we take partial derivatives of the Lagrangian.

a) **Derivative with respect to w**

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^m \alpha_i y_i x_i = 0$$

- So,

$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

- This shows that **the weight vector is a combination of training points.**

Apply Karush–Kuhn–Tucker (KKT) Conditions

b) Derivative with respect to b

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^m \alpha_i y_i = 0$$

- Thus,

$$\sum_{i=1}^m \alpha_i y_i = 0$$

Substitute w into the First Term

- The first term is:

$$\frac{1}{2} \| w \|^2$$

- Since,

$$w = \sum_{i=1}^m \alpha_i y_i x_i \text{ and } \| w \|^2 = w^T w$$

- Substitute w :

$$w^T w = \left(\sum_{i=1}^m \alpha_i y_i x_i \right)^T \left(\sum_{j=1}^m \alpha_j y_j x_j \right)$$

Substitute w into the First Term

- Expanding gives:

$$\sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

- Thus the first term becomes

$$\frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

Substitute w into the Second Term

- The second term is:

$$\sum_{i=1}^m \alpha_i [y_i(w^T x_i + b) - 1]$$

- Expand the Expression:

$$\sum_{i=1}^m \alpha_i y_i (w^T x_i) + \sum_{i=1}^m \alpha_i y_i b - \sum_{i=1}^m \alpha_i$$

- So the term becomes:

$$\sum_{i=1}^m \alpha_i y_i (w^T x_i) + b \sum_{i=1}^m \alpha_i y_i - \sum_{i=1}^m \alpha_i$$

Substitute w into the Second Term

- We have already derived,

$$\sum_{i=1}^m \alpha_i y_i = 0$$

- The middle term becomes: $b \times 0 = 0$
- Then, the expression becomes:

$$\sum_{i=1}^m \alpha_i y_i (w^T x_i) - \sum_{i=1}^m \alpha_i$$

- Now Substitute w :

$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

Substitute w into the Second Term

- Thus,

$$w^T x_i = \left(\sum_{j=1}^m \alpha_j y_j x_j \right)^T x_i$$

- which becomes,

$$\sum_{j=1}^m \alpha_j y_j (x_j^T x_i)$$

- Substituting into the equation produces the **double summation**:

$$\sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j) - \sum_{i=1}^m \alpha_i$$

Combine Both Terms

$$L = \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j) - \left(\sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j) - \sum_{i=1}^m \alpha_i \right)$$

- Now, it gives the **dual objective function**,

$$L = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

- The optimization depends only on **dot products between training points**, which later allows us to replace it with a **kernel function** $k(x, x')$.
- So, the final expression becomes:

$$L = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j k(x, x')$$

Key Idea

- The important part of this dual optimization is:

$$\sum_{i=1}^m \alpha_i y_i = 0$$

- It removes the **bias** b from the optimization problem.
- This is why the **dual formulation depends only on α_i** .
- The **primal problem** finds the hyperplane by minimizing $\|w\|^2$.
- The **dual problem** finds the best coefficients α_i that reconstruct w , i.e.,

$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

- By maximizing the dual function, we find the **optimal multipliers α_i** that define the hyperplane.

Final Dual Optimization Function

- The primal SVM problem minimizes the margin function with respect to w and b .
- When we form the Lagrangian and eliminate w and b , the resulting dual function depends only on α .
- According to Lagrange duality, this function must be **maximized with respect to α** , which gives the optimal support vectors.
- So, the expression will become:

$$\text{max}(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j k(x, x')$$

- Subject to

1. $\alpha_i \geq 0$

2. $\sum_{i=1}^m \alpha_i y_i = 0$

SVM Decision Boundary

- Once α_i are known, the **weight vector** w can be computed as:

$$w = \sum_{i=1}^m \alpha_i y_i x_i$$

- For nonlinear SVM using kernels, the **decision function** becomes:

$$f(x) = \sum_{i=1}^m \alpha_i y_i K(x_i, x_i') + b$$

- Here,
 - α_i is Lagrange multiplier obtained from solving the **dual optimization problem**.
 - $x_i \rightarrow$ test data point
 - x_i' is training data points
 - $K(x_i, x_i')$ is kernel function measuring similarity
 - b Bias term that shifts the decision boundary.

Determining the Bias b

- Support vectors lie **exactly on the margin boundary**, so they satisfy

$$y_i(w^T x_i + b) = 1$$

- Solving for b :

$$b = y_i - w^T x_i$$

- This can be computed using **any support vector**.
- Sometimes the average of several support vectors is used for better stability.

Final Decision Function

- For a new data point x , the classification rule is

$$f(x) = \sum_{i=1}^m \alpha_i y_i K(x_i, x_i') + b$$

- Decision rule:
 - if $f(x) > 0 \rightarrow \text{class } +1$
 - if $f(x) < 0 \rightarrow \text{class } -1$

Important Properties

1. Uses Only Support Vectors

- Since most $\alpha_i = 0$, the classifier depends only on **a few training points**.
- This makes SVM efficient.

2. Enables Nonlinear Classification

- Because the formula contains the **kernel function**

$$K(x_i, x_i')$$

- We can map data into **high-dimensional space** without explicitly computing the transformation.
- This technique is called the **kernel trick**.

Types of Support Vector Machine

- Based on the nature of the decision boundary, Support Vector Machines (SVM) can be divided into two main kinds:
 1. Linear SVM
 2. Non-Linear SVM

Linear SVM

- Linear SVMs use a linear decision boundary to separate the data points of different classes.
- When the data can be precisely linearly separated, linear SVMs are very suitable.
- This means that a single straight line (in 2D) or a hyperplane (in higher dimensions) can entirely divide the data points into their respective classes.
- A hyperplane that maximizes the margin between the classes is the decision boundary.

Non-Linear SVM

- It can be used to classify data when it cannot be separated into two classes by a straight line (in the case of 2D).
- By using kernel functions, non-linear SVMs can handle nonlinearly separable data.
- The original input data is transformed by these kernel functions into a higher-dimensional feature space where the data points can be linearly separated.
- A linear SVM is used to locate a nonlinear decision boundary in this modified space.

Advantages of Support Vector Machine

1. **High-Dimensional Performance:** SVM excels in high-dimensional spaces, making it suitable for image classification and gene expression analysis.
2. **Nonlinear Capability:** Utilizing kernel functions like RBF and polynomial; SVM effectively handles nonlinear relationships.
3. **Outlier Resilience:** The soft margin feature allows SVM to ignore outliers, enhancing robustness in spam detection and anomaly detection.
4. **Binary and Multiclass Support:** SVM is effective for both binary classification and multiclass classification suitable for applications in text classification.
5. **Memory Efficiency:** It focuses on support vectors making it memory efficient compared to other algorithms.

Disadvantages of Support Vector Machine

1. **Slow Training:** SVM can be slow for large datasets, affecting performance in SVM in data mining tasks.
2. **Parameter Tuning Difficulty:** Selecting the right kernel and adjusting parameters like C requires careful tuning, impacting SVM algorithms.
3. **Noise Sensitivity:** SVM struggles with noisy datasets and overlapping classes, limiting effectiveness in real-world scenarios.
4. **Limited Interpretability:** The complexity of the hyperplane in higher dimensions makes SVM less interpretable than other models.
5. **Feature Scaling Sensitivity:** Proper feature scaling is essential, otherwise SVM models may perform poorly.

Question 1

Identify the Hyperplane and Margin for a given classifier:

$$w = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, b = -10$$

- a) Write the equation of the hyperplane.
- b) Compute the geometric margin.
- c) Find the distance of point $x = (4, 2)$ from the hyperplane.

Solution 1

a) Hyperplane equation:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

$$3x_1 + 4x_2 - 10 = 0$$

b) Margin:

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}$$

$$\|\mathbf{w}\| = \sqrt{3^2 + 4^2} = 5$$

$$\text{margin} = \frac{2}{5} = 0.4$$

c) Distance of point (4, 2)

$$\frac{|3(4) + 4(2) - 10|}{\|\mathbf{w}\|} = \frac{|12 + 8 - 10|}{5} = \frac{10}{5} = 2$$

Question 2

Given:

$$w = (2, -1), b = -3$$

Dataset:

x_1	x_2	y
3	2	+1
1	1	-1

- (a) Compute functional margin of each point.
- (b) Compute geometric margin of dataset.

Functional Margin

- The **functional margin** of a training example (x_i, y_i) is:

$$\gamma_i = y_i(w^T x_i + b)$$

- It Measures **how correctly and confidently a point is classified** by the hyperplane.
- It depends on the **scale of w and b** .

Important Property:

- If we multiply the parameters by a constant k :

$$w' = kw, b' = kb$$

then

$$\gamma_i' = k\gamma_i$$

- So, the functional margin changes with scaling, even though the hyperplane stays the same.
- Therefore, it does not represent actual distance from the hyperplane.

Geometric Margin

- The **geometric margin** is the **perpendicular distance** from the point to the hyperplane.

$$\text{Geometric margin} = \frac{y_i(w^T x_i + b)}{\|w\|}$$

- It measures the **true Euclidean distance** from the data point to the decision boundary.
- It is **Independent of scaling** of w and b .
- If w and b are multiplied by k :

$$\gamma'_i = \frac{k(y_i(w^T x_i + b))}{k \|w\|} = \gamma_i$$

- So the geometric margin **remains unchanged**.

Solution 2

Functional margin:

$$y_i(w^T x_i + b)$$

For (3, 2),

$$((2 * 3) + (-1 * 2) - 3) = 6 - 2 - 3 = 1$$

Multiply by y :

$$(+1)(1) = 1$$

For (1, 1),

$$((2 * 1) + (-1 * 1) - 3) = 2 - 1 - 3 = -2$$

Multiply by y :

$$(-1)(-2) = 2$$

Solution 2

Geometric margin:

$$\| w \| = \sqrt{2^2 + (-1)^2} = \sqrt{5}$$

For data point 1:

$$\text{Geometric margin} = \frac{2}{\| w \|} = \frac{2}{\sqrt{5}}$$

For data point 2:

$$\text{Geometric margin} = \frac{1}{\sqrt{5}}$$

Dataset margin will be,

$$\text{Dataset margin} = \min\left(\frac{1}{\sqrt{5}}, \frac{2}{\sqrt{5}}\right) = \frac{1}{\sqrt{5}}$$

Question 3

Given hyperplane is:

$$x_1 + 2x_2 - 4 = 0$$

Points:

x_1	x_2	y
2	1	+1
4	0	+1
1	1	-1

Identify which points are support vectors.

Solution 3

Compute:

$$y(w^T x + b)$$

For (2, 1):

$$2 + 2(1) - 4 = 0$$

Functional margin = 0 $\rightarrow$ ON hyperplane $\rightarrow$ Support vector

For (4, 0):

$$4 + 0 - 4 = 0 \rightarrow \text{Support vector}$$

For (1, 1):

$$1 + 2 - 4 = -1$$

Multiply by $y = (-1)$:

$$(-1)(-1) = 1 \rightarrow \text{Not support vector (margin} > 0)$$

Question 4

Find Optimal Hyperplane for two support vectors:

$$x^{(1)} = (2, 2), y = +1$$

$$x^{(2)} = (0, 0), y = -1$$

Assume they lie exactly on margin.

Find:

- a) w
- b) b
- c) margin

Solution 4

Since support vectors satisfy:

$$w^T x + b = 1 \text{ and } w^T x + b = -1$$

For (2, 2):

$$2w_1 + 2w_2 + b = 1$$

For (0, 0):

$$b = -1$$

Substitute:

$$2w_1 + 2w_2 - 1 = 1$$

$$2w_1 + 2w_2 = 2$$

$$w_1 + w_2 = 1$$

One possible symmetric solution: $w_1 = w_2 = 0.5$

Solution 4

Margin:

$$\| w \| = \sqrt{0.5^2 + 0.5^2} = \sqrt{0.5}$$

$$\text{Margin} = \frac{2}{\sqrt{0.5}} = 2\sqrt{2}$$

Question 5

Minimize:

$$\frac{1}{2} || w ||^2$$

Subject to:

$$y_i(w^T x_i + b) \geq 1$$

Given dataset:

x_1	x_2	y
1	1	+1
-1	-1	-1

Find optimal w and b .

Solution 5

Point 1 : $x = (1, 1)$, $y = +1$

$$\begin{aligned} (+1)(w_1(1) + w_2(1) + b) &= 1 \\ w_1 + w_2 + b &= 1 \end{aligned}$$

Point 2 : $x = (-1, -1)$, $y = -1$

$$\begin{aligned} (-1)(w_1(-1) + w_2(-1) + b) &= 1 \\ (-1)(-w_1 - w_2 + b) &= 1 \\ w_1 + w_2 - b &= 1 \end{aligned}$$

Subtract equations:

$$\begin{aligned} (w_1 + w_2 + b) - (w_1 + w_2 - b) &= 0 \\ 2b &= 0 \\ b &= 0 \end{aligned}$$

Solution 5

Substitute $b = 0$

$$w_1 + w_2 = 1$$

Minimize $\|w\|^2$

$$\|w\|^2 = w_1^2 + w_2^2$$

subject to

$$w_1 + w_2 = 1$$

The minimum occurs when the weights are equal:

$$w_1 = w_2$$

Thus

$$w_1 = w_2 = \frac{1}{2} = 0.5$$

Question 6

Given two vectors:

$$x = (2, 1) \text{ and } z = (3, 4)$$

The linear kernel is defined as: $K(x, z) = x \cdot z$

- (a) Compute the linear kernel value between x and z .
- (b) Interpret the result in terms of vector similarity.

Solution 6

$$x \cdot z = (2)(3) + (1)(4) = 6 + 4 = 10$$

So,

$$K(x, z) = 10$$

A larger value indicates higher similarity between the vectors.

Question 7

Given vectors:

$$x = (1, 2) \text{ and } z = (3, 1)$$

The polynomial kernel of degree 2 is defined as:

$$K(x, z) = (x \cdot z + 1)^2$$

Compute the polynomial kernel value.

Solution 7

First compute the dot product:

$$x \cdot z = (1)(3) + (2)(1) = 3 + 2 = 5$$

Now apply the kernel formula:

$$K(x, z) = (5 + 1)^2 = 6^2 = 36$$

Thank You!